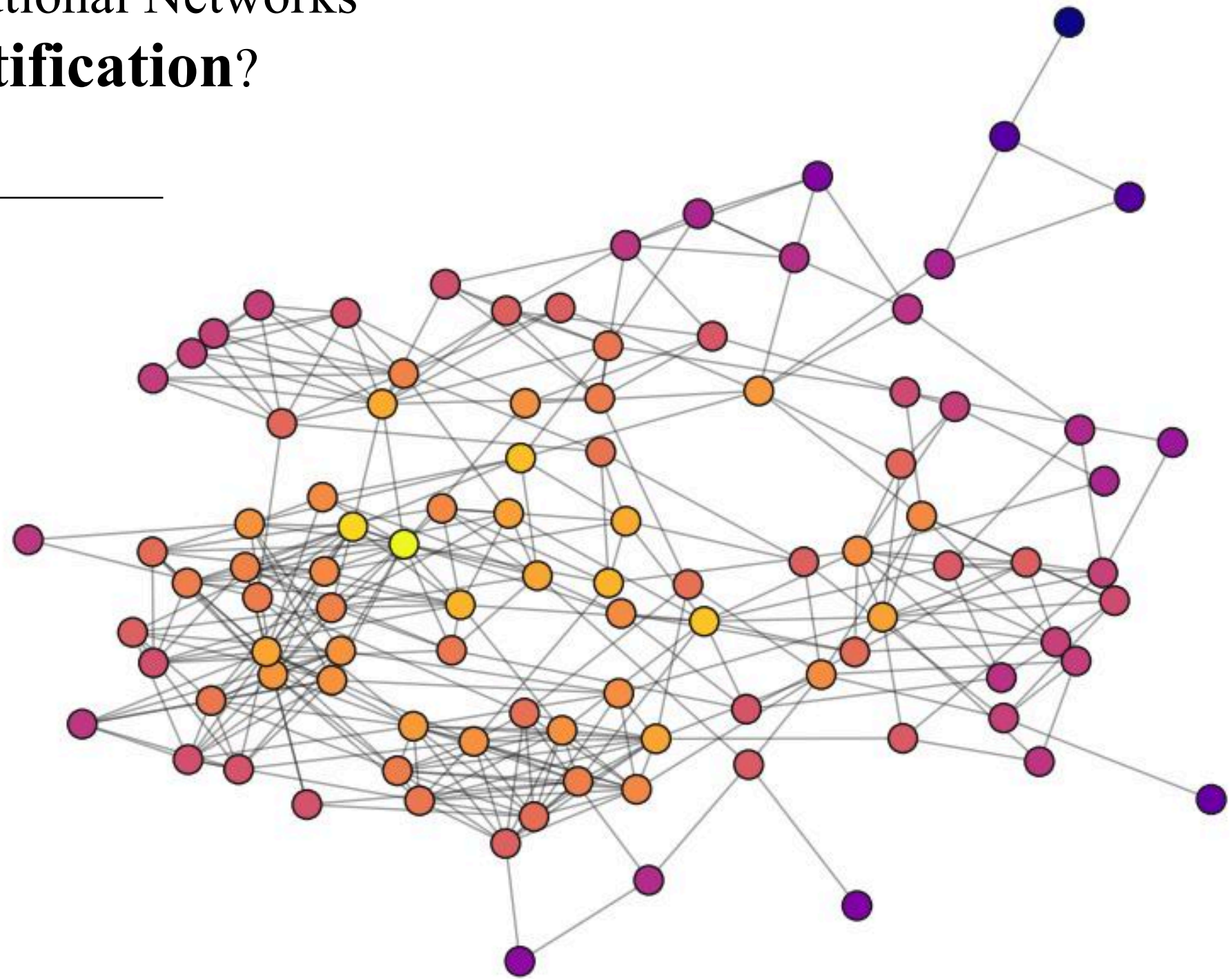


Can **Graph Neural Networks** be better than Convolutional Networks **for tool wear identification?**



Nishit Singh
Siddharth Anand Bharadwaj

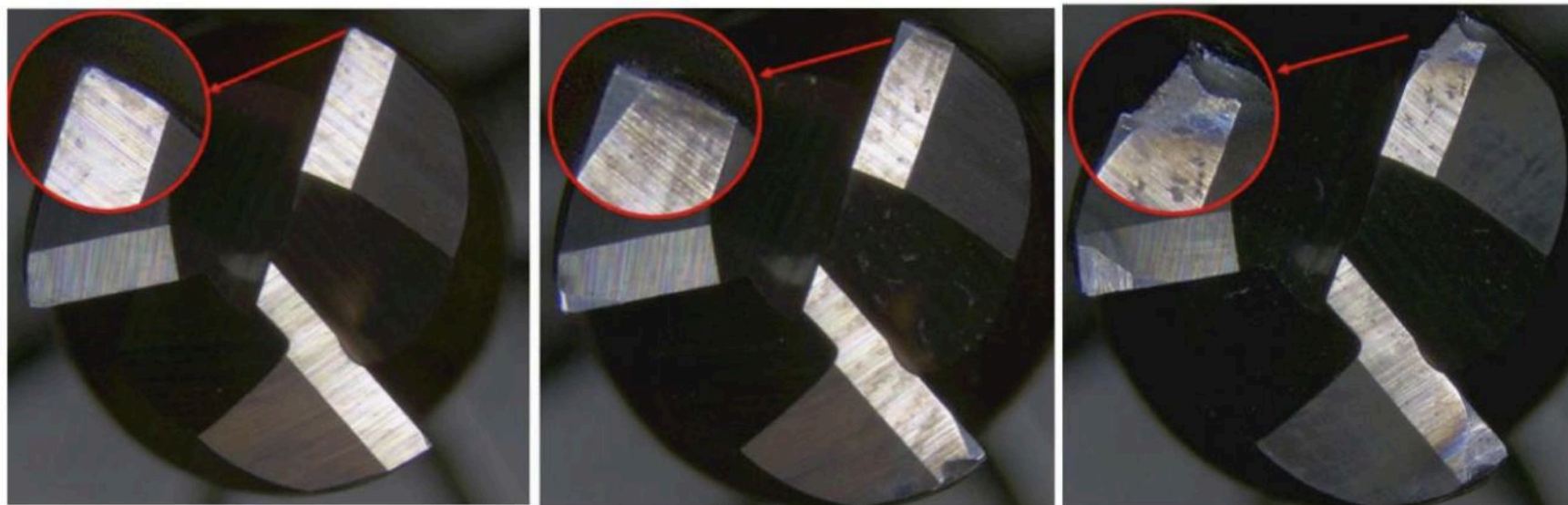
NTMP, 2026

Where do CNNs fail?

CNNs like ResNet require **thousands of samples** in the training dataset to approach high accuracy.

Tool wear, by virtue, is an occurrence that occurs at a rate much slower than required for CNN requirements.

Graph Neural Networks display higher sample efficiency.

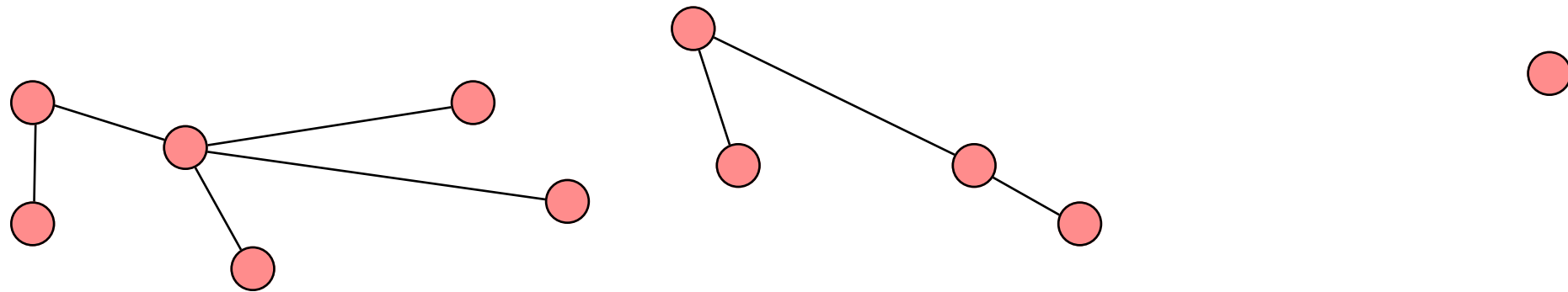


(a) Second milling stage (b) Sixth milling stage (c) Tenth milling stage

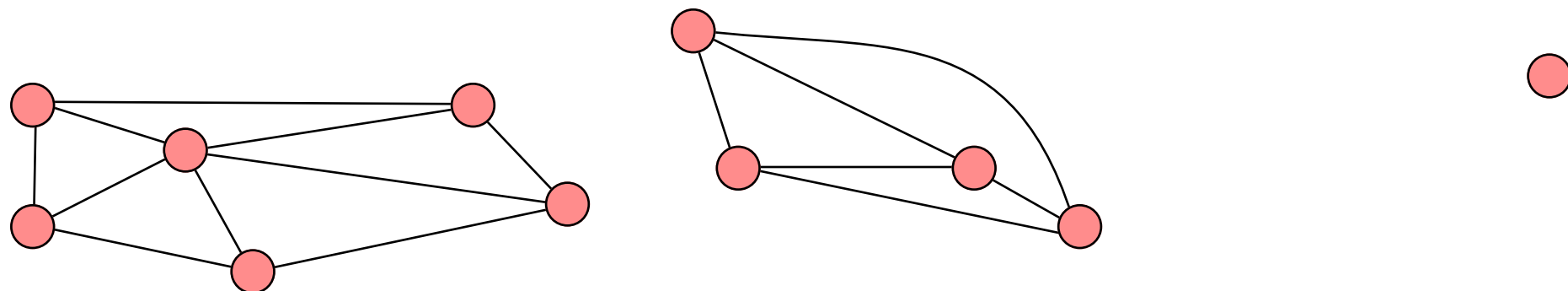
What is a Graph Neural Network (GNN)?

Imagine a toy problem : Who is likely to become friends in a big college setting?

Form a graph : Nodes are people, two people are connected by an edge if friends.



Update each node iteratively : Friends are more likely to become friends of friends and so on.

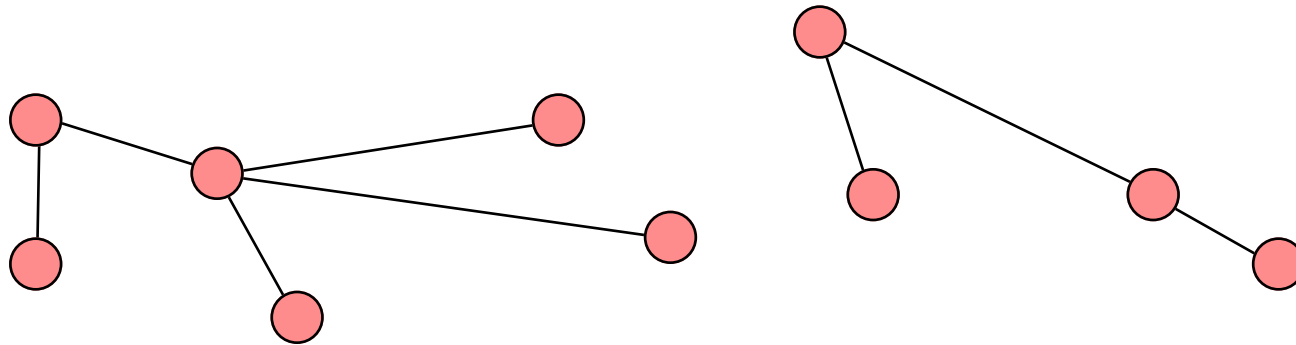


Highly mutual clusters all become friends

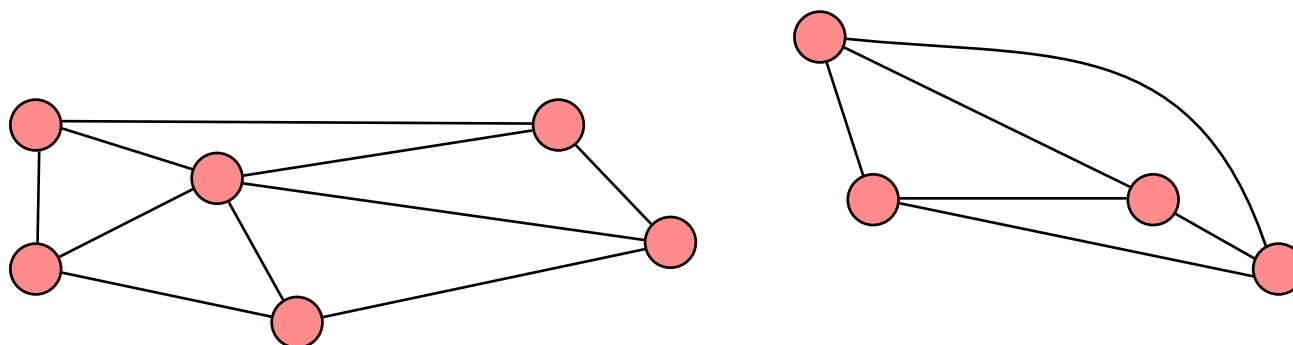
Loners don't :(

What is a Graph Neural Network (GNN)?

Formally : Each node V in a graph $G = (V, E)$ with E edges updates its state V_i according to the rule $V_{(i+1)} = f(V_i, V_{\text{neighborhood}})$



$G = (11, 8)$



$G = (11, 16)$

Zhi et al. found a way to exploit GNNs for tool wear classification

IOP Publishing
Meas. Sci. Technol. **32** (2021) 064006 (10pp)

Measurement Science and Technology
<https://doi.org/10.1088/1361-6501/abe0d9>

An edge-labeling graph neural network method for tool wear condition monitoring using wear image with small samples

Gaofeng Zhi¹, Dedao He², Weifang Sun¹ , Yuqing Zhou¹ , Xiaoming Pan¹ and Chen Gao³

¹ College of Mechanical and Electrical Engineering, Wenzhou University, Wenzhou, People's Republic of China

² Technology Center, Wenzhou Ruiming Industrial Co., Ltd, Wenzhou, People's Republic of China

³ School of Mechatronics and Transportation, Jiaxing Nanyang Polytechnic Institute, Jiaxing, People's Republic of China

E-mail: zhouyq@wzu.edu.cn and gaochen_1993@163.com

Received 16 October 2020, revised 21 January 2021

Accepted for publication 28 January 2021

Published 14 April 2021



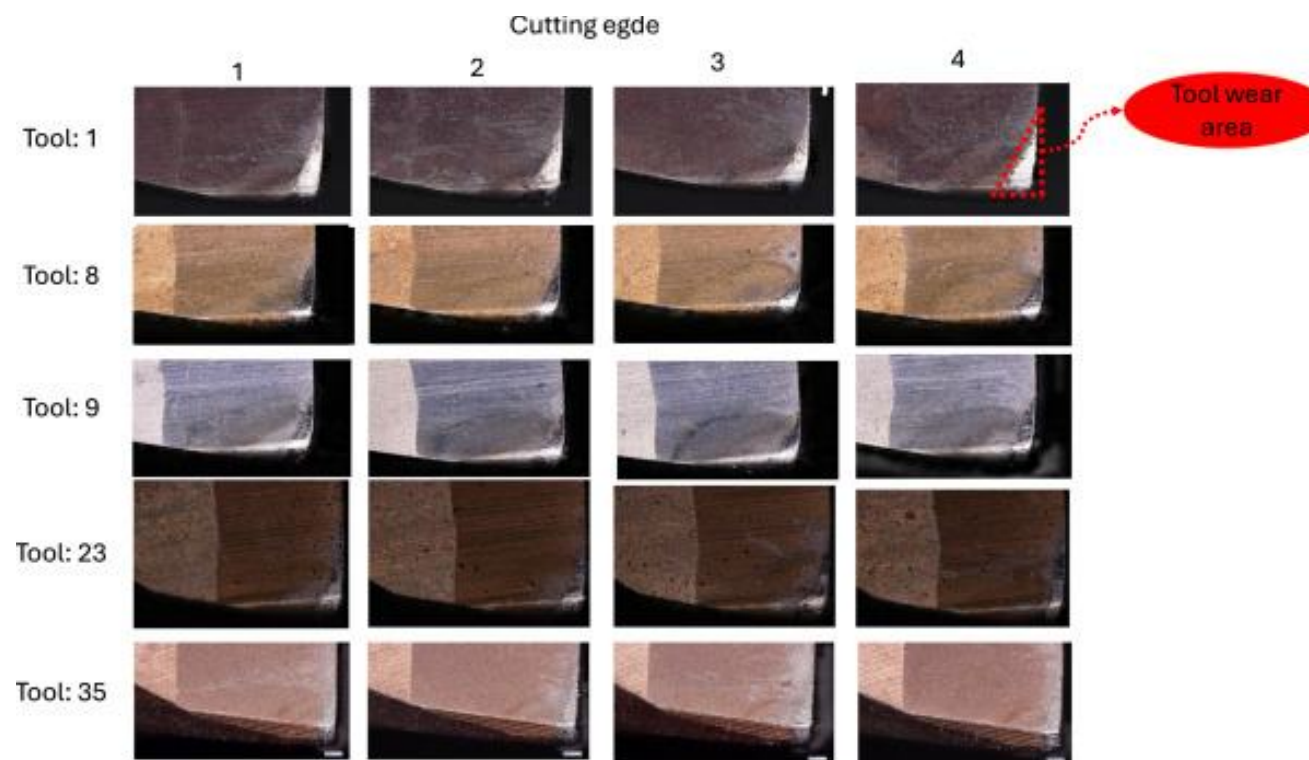
Abstract

Tool wear condition monitoring (TCM) is of great significance to ensure manufacturing quality in milling processes, and the development of deep learning (DL) in recent years has led to increasing interest in DL-based TCM methods. However, most of these DL-based methods rely on large training samples to achieve good performances, which is expensive. In this paper, a new TCM method based on an edge-labeling graph neural network (EGNN) is proposed for small training datasets. First, the tool wear image is input into a convolution neural network (CNN) to extract features and obtain the features of the training samples. A fully connected graph is established based on these features, and the values of the edge labels are obtained by updating the nodes and edge features in the fully connected graph. Finally, the tool wear condition is predicted through the sample label of the support set and the predicted value of the edge connected with the query sample using a weighted voting method. The effectiveness of the proposed EGNN-based TCM method was demonstrated by its application to milling TCM experiments, and the results indicated that the proposed method outperformed three state-of-the-art methods (CNN, AlexNet, and ResNet) with small samples.

TLDR : GNNs when trained on image classification for Tool Condition Monitoring (TCM) are more sample efficient than CNNs.

Recipe to build a GNN for image classification

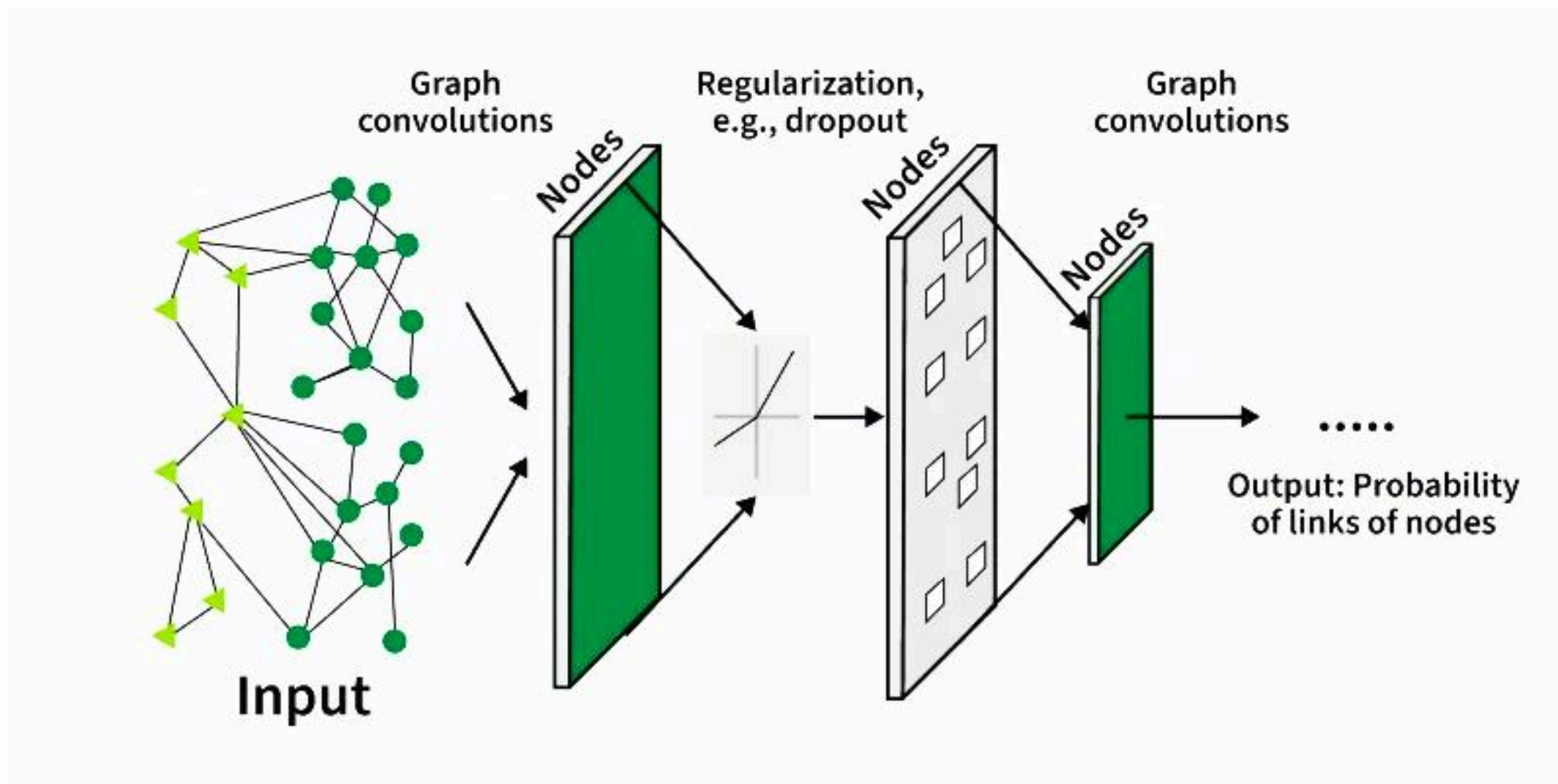
Build a Dataset : Form a dataset of pictures of tools in various points in their lifecycles.



Build a ground truth : Assign each image a ground truth value (how worn is it?) and its relation to other images (how similar is it to this other image?)

Recipe to build a GNN for image classification

Train a GNN to update itself : Form and update it's own edges and nodes according to some plan so that it can place new nodes (query nodes) in the correct position.



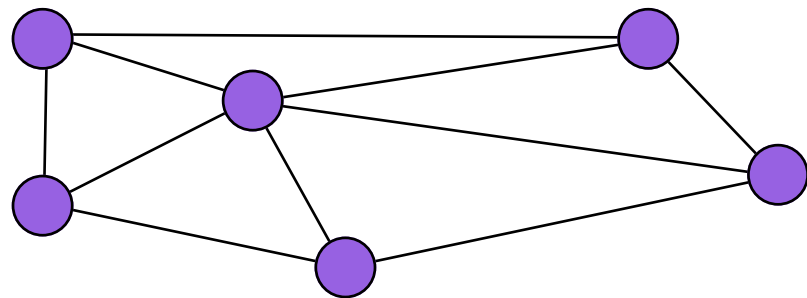
Ground Truth / Labelling

Each edge has a ground truth array :

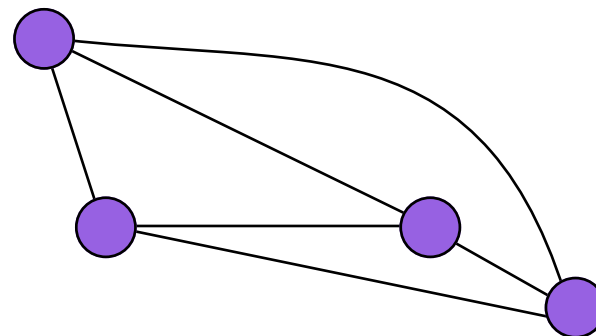
Thus say for edge connecting the i^{th} and j^{th} node, $e_{ij}^{(0)} = \begin{cases} [1, 0] & \text{same class} \\ [0, 1] & \text{different class} \end{cases}$

Which is equivalent to $e_{ij} = [\text{how likely same class}, \text{how likely different class}]$

For an edge which the model is completely unsure of, it will output $[0.5, 0.5]$.



Low tool wear group



Medium tool wear group



Worn tool group

Update rules

Each node updates according to the weights of its edges.

$$v_i^{(\ell)} = f_v^{(\ell)} \left(\left[\sum_j \tilde{e}_{ij1}^{(\ell-1)} v_j^{(\ell-1)} \parallel \sum_j \tilde{e}_{ij2}^{(\ell-1)} v_j^{(\ell-1)} \right] \right)$$

which, in simpler words is just

new_node = f("sum of similar neighbors", "sum of dissimilar neighbors")

Normalising the edges (so that they are squished between 0 and 1),

$$\tilde{e}_{ijd} = \frac{e_{ijd}}{\sum_k e_{ikd}}$$

We have our node update rule.

Update rules

Each edge updates according to the weights of its nodes,

$$\bar{e}_{ij1}^{(\ell)} = \frac{f_e(v_i^{(\ell)}, v_j^{(\ell)}) \cdot e_{ij1}^{(\ell-1)}}{\sum_k f_e(v_i^{(\ell)}, v_k^{(\ell)}) \cdot e_{ik1}^{(\ell-1)}} \quad \bar{e}_{ij2}^{(\ell)} = \frac{(1 - f_e(v_i^{(\ell)}, v_j^{(\ell)})) \cdot e_{ij2}^{(\ell-1)}}{\sum_k (1 - f_e(v_i^{(\ell)}, v_k^{(\ell)})) \cdot e_{ik2}^{(\ell-1)}}$$

where $f_e(v_i, v_j)$ = similarity between nodes,

the first equation describing increasing probability that they are in the same class if they're similar

the second equation describing increasing different class probability if they are dissimilar.

**Nodes update according to edges, edges update according to nodes.
Co-evolution.**

Running a query on softmax attention (SMAT), each node votes for the input node (similar to me, dissimilar to me), and the importance of the vote (weight) is decided by the edge strength.

Results

Table 4. Comparison of the accuracy of the different methods in the experiments.

Method	CNN	AlexNet	ResNet	Proposed model
Accuracy rate	73.2%	70.9%	64.3%	78.8%

Table 5. Classification accuracy with different proportion of samples.

Proportion of samples	50%	60%	70%
Training sample size	19	23	26
Testing sample size	18	14	11
Classification accuracy of the proposed method	76.8%	78.8%	80.2%